

Universidad Autónoma de la Ciudad de México

Licenciatura en Ingeniería de Software

Arquitectura de Computadoras

Proyecto 01

Conversión de Imagen BMP a Escala de Grises en Ensamblador
x64

Alumno: Edson Joel Carrera Avila

Grupo: 302

Profesor: Prof. Mtro. Omar Nieto Crisóstomo

Fecha: 24 de mayo de 2026

Índice

1. Introducción	2
2. Organización del proyecto	3
3. Anatomía del archivo BMP	5
3.1. Bitmap File Header (14 bytes)	5
3.2. DIB Header / BITMAPINFOHEADER (40 bytes)	5
4. Código fuente	6
4.1. Programa principal (main.cpp)	6
4.2. Módulo ensamblador (grayscale.asm)	7
5. Instrucciones x86/x64 utilizadas	8
5.1. Propósito y aritmética	8
5.2. Control de flujo	8
6. El rol de los registros	8
6.1. Convención de llamadas Microsoft x64	8
6.2. Registros internos	9
7. Ejemplo de ejecución	9
7.1. Imagen de entrada	9
7.2. Salida en consola	10
7.3. Imagen de salida en escala de grises	10
7.4. Inspección hexadecimal de las cabeceras	11
8. Repositorio GitHub	11
9. Conclusiones	11

1. Introducción

Este proyecto consiste en **leer una imagen en formato BMP de 24 bits**, convertir cada uno de sus píxeles a un tono de gris equivalente usando una rutina escrita en **ensamblador x64 (MASM)**, y escribir el resultado en un nuevo archivo BMP. El programa principal en C++ se encarga de abrir el archivo, validar la cabecera, recorrer las filas de píxeles, invocar la rutina ensambladora con cada fila y finalmente guardar el archivo de salida.

¿Qué es el formato BMP?

BMP (*BitMap*), también llamado **DIB** (*Device-Independent Bitmap*), es un formato de archivo desarrollado por Microsoft para almacenar mapas de bits sin compresión. Su estructura básica consta de cuatro bloques contiguos:

1. **Bitmap File Header** (BMPHeader, 14 bytes): identifica el archivo con la firma ASCII "BM" (0x4D42) y declara el tamaño total y el desplazamiento (`offset_data`) hasta donde comienzan los píxeles [4].
2. **Bitmap Info Header** (DIBHeader, normalmente 40 bytes para BITMAPINFOHEADER): describe las dimensiones (`width`, `height`), el número de bits por píxel (`bit_count`), el tipo de compresión y otras propiedades [2].
3. **Color Palette** (opcional): tabla de colores presente cuando `bit_count` es menor a 24. En imágenes de 24 bits **no existe paleta**, los píxeles guardan sus colores explícitamente.
4. **Pixel Array**: los datos de la imagen propiamente dichos.

¿Cómo se almacenan los píxeles en un BMP de 24 bits?

En un BMP de 24 bits cada píxel ocupa 3 bytes y se almacena en el orden **BGR** (no RGB): primero el byte del canal Azul, luego el Verde y finalmente el Rojo [4]. Este orden invertido proviene de la representación natural de un entero de 32 bits en arquitecturas *little-endian* (Intel/AMD), donde los bytes menos significativos quedan en las direcciones más bajas.

Además, el estándar BMP exige que cada fila (*scan line*) tenga un tamaño en bytes **múltiplo de 4** [3]. Si el ancho en píxeles multiplicado por 3 no es divisible entre 4, se añaden bytes de **relleno** (*padding*) al final de la fila. El cálculo del relleno es:

$$\text{padding} = (4 - (\text{width} \times 3 \bmod 4)) \bmod 4 \quad (1)$$

$$\text{row_stride} = \text{width} \times 3 + \text{padding} \quad (2)$$

Por ejemplo, una imagen de 100 píxeles de ancho tiene filas de $100 \times 3 = 300$ bytes, que ya es múltiplo de 4, por lo que no necesita relleno. Pero una imagen de 33 píxeles de

ancho tendría filas de $33 \times 3 = 99$ bytes, y necesita un byte de relleno para alcanzar los 100 bytes [4].

Otra peculiaridad: las filas se almacenan **de abajo hacia arriba** cuando la altura (`biHeight`) es positiva. Es decir, la primera fila del archivo corresponde a la fila más baja de la imagen. Si `biHeight` es negativa, las filas se almacenan de arriba hacia abajo (*top-down DIB*) [2].

¿Por qué los pesos 0,299, 0,587, 0,114?

Los pesos 0,299, 0,587 y 0,114 provienen del estándar **ITU-R BT.601** de la Unión Internacional de Telecomunicaciones.

$$Y' = 0,299 R' + 0,587 G' + 0,114 B' \quad (3)$$

Estos coeficientes reflejan la **sensibilidad relativa del ojo humano** a cada color: el verde aporta cerca del 59 % de la luminosidad percibida, el rojo el 30 %, y el azul apenas el 11 %. Por eso, un simple promedio aritmético $\frac{R+G+B}{3}$ produce imágenes que se ven incorrectas (cielos demasiado claros, vegetación demasiado oscura).

¿Por qué multiplicar por 256 los pesos?

El procesador trabaja muy eficientemente con **aritmética entera**, mientras que las multiplicaciones en punto flotante son notoriamente más lentas. Una optimización clásica consiste en **escalar los pesos por una potencia de 2** para evitar el uso de la FPU:

$$Y \approx \frac{29 B + 150 G + 77 R}{256} \quad (4)$$

donde $29 \approx 0,114 \times 256$, $150 \approx 0,587 \times 256$ y $77 \approx 0,299 \times 256$. La suma de los pesos enteros es exactamente $29 + 150 + 77 = 256 = 2^8$, lo que permite reemplazar la división final por un **desplazamiento a la derecha** de 8 bits (`SHR r11d, 8`), una de las instrucciones más rápidas del procesador [5]. Esta es exactamente la estrategia usada en `grayscale.asm`.

2. Organización del proyecto

El proyecto se compila como una aplicación de consola **x64** en Visual Studio con el conjunto de herramientas **v143**. La plataforma debe ser obligatoriamente **x64** porque las rutinas usan la convención de llamadas Microsoft x64 (`RCX`, `RDX`, `R8`) [6].

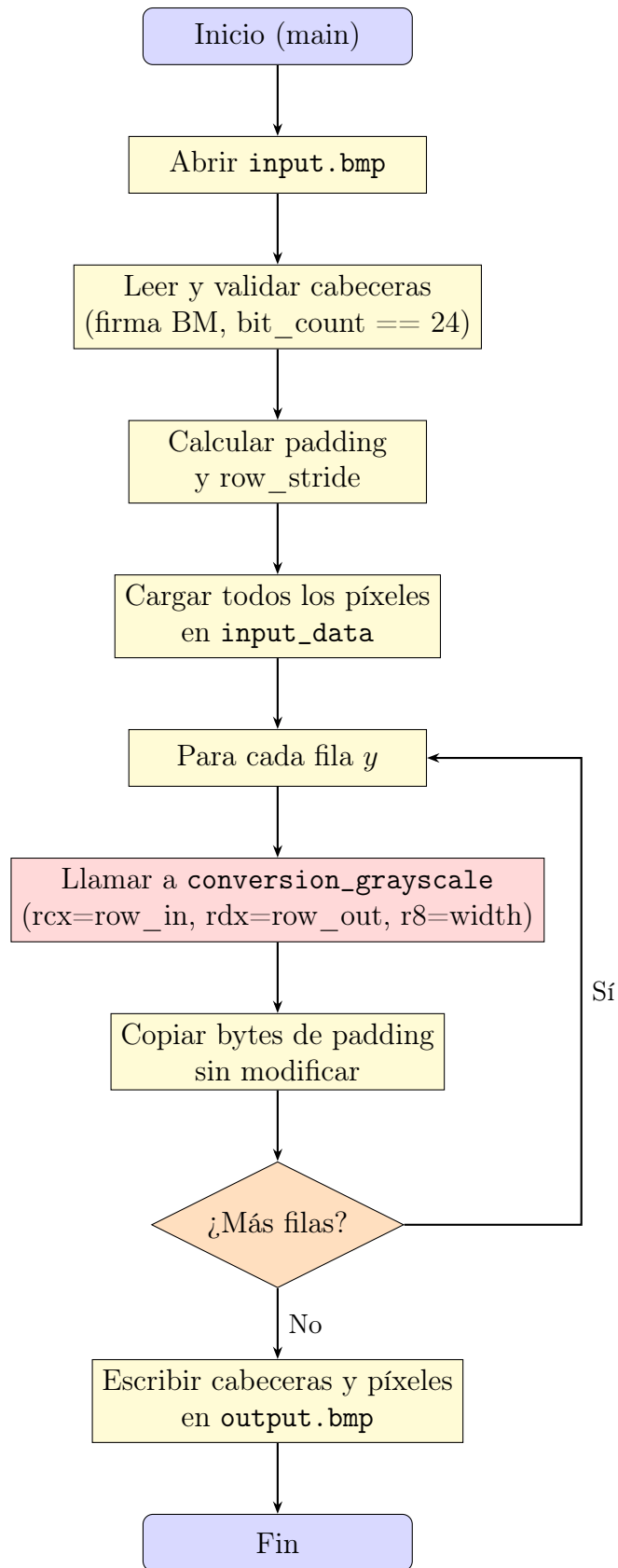


Figura 1: Diagrama de flujo del procesamiento BMP fila por fila

3. Anatomía del archivo BMP

3.1. Bitmap File Header (14 bytes)

Offset	Tamaño	Campo	Descripción
0x00	2	file_type	Firma "BM" (0x4D42)
0x02	4	file_size	Tamaño total del archivo en bytes
0x06	2	reserved1	Reservado, siempre 0
0x08	2	reserved2	Reservado, siempre 0
0x0A	4	offset_data	Desplazamiento al inicio del array de píxeles

Cuadro 1: Estructura del BMPHeader según la especificación de Microsoft [1]

3.2. DIB Header / BITMAPINFOHEADER (40 bytes)

Offset	Tamaño	Campo	Descripción
0x0E	4	size	Tamaño de la cabecera (típicamente 40)
0x12	4	width	Ancho en píxeles
0x16	4	height	Alto en píxeles (positivo = bottom-up)
0x1A	2	planes	Siempre 1
0x1C	2	bit_count	Bits por píxel (24 en este proyecto)
0x1E	4	compression	0 = sin compresión (BI_RGB)
0x22	4	size_image	Tamaño del array de píxeles
0x26	4	x_pixels_per_meter	Resolución horizontal
0x2A	4	y_pixels_per_meter	Resolución vertical
0x2E	4	colors_used	Colores usados (0 = todos)
0x32	4	colors_important	Colores importantes

Cuadro 2: Estructura del DIBHeader (BITMAPINFOHEADER) [2]

El uso de #pragma pack(push, 1)

El programa principal usa la directiva `#pragma pack(push, 1)` para garantizar que el compilador **no inserte bytes de relleno** entre los campos de las estructuras. Sin esta directiva, el compilador podría alinear los campos a fronteras naturales de 4 u 8 bytes para optimizar accesos a memoria [7], lo que rompería completamente el mapeo binario con el archivo BMP en disco: por ejemplo, el campo `file_size` debe estar exactamente en el offset 0x02, no en 0x04 como ocurriría con alineación natural.

4. Código fuente

4.1. Programa principal (main.cpp)

El programa principal se encarga de toda la lógica de archivos (apertura, validación, padding, escritura) y delega a ensamblador únicamente la transformación píxel a píxel.

```

1 #pragma pack(push, 1)
2 struct BMPHeader {
3     uint16_t file_type{0x4D42}; // Firma 'BM'
4     uint32_t file_size{0};
5     uint16_t reserved1{0};
6     uint16_t reserved2{0};
7     uint32_t offset_data{0};
8 };
9
10 struct DIBHeader {
11     uint32_t size{0};
12     int32_t width{0};
13     int32_t height{0};
14     uint16_t planes{1};
15     uint16_t bit_count{0};
16     uint32_t compression{0};
17     uint32_t size_image{0};
18     int32_t x_pixels_per_meter{0};
19     int32_t y_pixels_per_meter{0};
20     uint32_t colors_used{0};
21     uint32_t colors_important{0};
22 };
23 #pragma pack(pop)

```

Listing 1: Definición de las cabeceras BMP empaquetadas a 1 byte

```

1 int width = dib_header.width;
2 int height = abs(dib_header.height);
3
4 int row_width_bytes = width * 3;
5 int padding = (4 - (row_width_bytes % 4)) % 4;
6 int row_stride = row_width_bytes + padding;
7
8 vector<uint8_t> input_data(row_stride * height);
9 file.read(reinterpret_cast<char*>(input_data.data()), input_data.size())
10 ;
11 vector<uint8_t> output_data(input_data.size());
12
13 for (int y = 0; y < height; ++y) {
14     uint8_t* row_in = input_data.data() + (y * row_stride);
15     uint8_t* row_out = output_data.data() + (y * row_stride);
16
17     conversion_grayscale(row_in, row_out, width);
18
19     // Conservar los bytes de padding originales
20     for (int p = 0; p < padding; ++p) {

```

```

21     row_out[row_width_bytes + p] = row_in[row_width_bytes + p];
22 }
23 }

```

Listing 2: Cálculo del padding y procesamiento fila por fila

4.2. Módulo ensamblador (grayscale.asm)

El módulo ensamblador implementa la conversión píxel a píxel siguiendo la fórmula entera $\frac{29B+150G+77R}{256}$.

```

1  conversion_grayscale PROC
2      test r8, r8                ; hay píxeles que procesar?
3      jz    fin                 ; si no, salir
4
5  bucle:
6      ; --- PASO 1: cargar los 3 canales (BGR) del píxel actual ---
7      movzx r9d, byte ptr [rcx]  ; B (azul, offset 0)
8      movzx r10d, byte ptr [rcx+1] ; G (verde, offset 1)
9      movzx r11d, byte ptr [rcx+2] ; R (rojo, offset 2)
10
11     ; --- PASO 2: multiplicar cada canal por su peso ---
12     imul r9d, 29                ; B * 29
13     imul r10d, 150              ; G * 150
14     imul r11d, 77              ; R * 77
15
16     ; --- PASO 3: sumar los tres productos ---
17     add r11d, r10d              ; R*77 + G*150
18     add r11d, r9d               ; + B*29
19
20     ; --- PASO 4: dividir por 256 (shift derecho de 8 bits) ---
21     shr r11d, 8                 ; gris = (29B + 150G + 77R) >> 8
22
23     ; --- PASO 5: escribir el mismo gris en los 3 canales de salida ---
24     mov byte ptr [rdx], r11b
25     mov byte ptr [rdx+1], r11b
26     mov byte ptr [rdx+2], r11b
27
28     ; --- PASO 6: avanzar al siguiente píxel (3 bytes) ---
29     add rcx, 3
30     add rdx, 3
31
32     ; --- PASO 7: decrementar contador y repetir ---
33     dec r8
34     jnz bucle
35
36 fin:
37     ret
38 conversion_grayscale ENDP

```

Listing 3: Rutina conversion_grayscale completa

5. Instrucciones x86/x64 utilizadas

5.1. Propósito y aritmética

Instrucción	Operación
TEST	Realiza una AND lógica entre dos operandos sin almacenar el resultado; sólo afecta banderas. Se usa para detectar si R8 es cero al inicio [5].
MOVZX	<i>Move with Zero-eXtend</i> : copia un byte/word a un registro más grande rellenando los bits superiores con ceros. Imprescindible antes de IMUL para evitar interpretar los bits altos como basura.
IMUL	<i>Integer Multiply</i> : multiplicación entera con signo. Aquí se usa con un inmediato como segundo operando (forma de 3 operandos).
ADD	Suma entera (acumula los productos parciales).
SHR	<i>Shift Right</i> : desplaza los bits a la derecha. Una división por 2^n se reduce a un SHR n, mucho más rápido que una división entera.
MOV	Mueve un byte de un registro a memoria (escritura del píxel de salida).

Cuadro 3: Instrucciones aritméticas y de movimiento

5.2. Control de flujo

Instrucción	Operación
JZ	<i>Jump if Zero</i> : salta si la bandera ZF está activada (resultado anterior fue cero).
DEC	Decrementa un registro en 1 y actualiza ZF si el resultado es cero.
JNZ	<i>Jump if Not Zero</i> : salta si ZF no está activada. Aquí mantiene vivo el bucle.
RET	Retorna al llamador (main.cpp).

Cuadro 4: Instrucciones de control de flujo del bucle

6. El rol de los registros

6.1. Convención de llamadas Microsoft x64

A diferencia de la *cdecl* de 32 bits (parámetros en la pila), en x64 los primeros cuatro argumentos enteros viajan en registros específicos [6]:

Registro	Parámetro	Descripción
RCX	uint8_t* input	Puntero a la fila de entrada (BGR)
RDX	uint8_t* output	Puntero a la fila de salida (gris)
R8	uint64_t pixel_count	Número de píxeles a procesar (= ancho)

Cuadro 5: Mapa de parámetros para la convención Microsoft x64

6.2. Registros internos

Registro	Subregistro 32-bit	Rol
R9	R9D	Almacena el canal B extendido a 32 bits. Tras <code>IMUL 29</code> , contiene $29B$.
R10	R10D	Almacena el canal G . Tras <code>IMUL 150</code> , contiene $150G$.
R11	R11D / R11B	Acumulador final del cálculo. R11D guarda la suma ponderada y, tras <code>SHR 8</code> , su byte bajo R11B contiene el valor de gris $[0-255]$.

Cuadro 6: Uso de los registros generales dentro de la rutina

¿Por qué R11B es exactamente el valor de gris?

Después de `IMUL` cada producto cabe en menos de 16 bits ($255 \times 150 = 38250 \approx 2^{16}$). La suma de los tres productos cabe holgadamente en 32 bits ($255 \times 256 = 65280$). Al dividir entre 256 con `SHR 8`, el resultado está garantizado en el rango $[0, 255]$, por lo que su **byte bajo** R11B es directamente el valor de gris a escribir.

7. Ejemplo de ejecución

7.1. Imagen de entrada

Como caso de prueba se utiliza una fotografía en color, guardada como BMP de 24 bits con cualquier editor (Paint, GIMP, Photoshop) en la ruta `src/input.bmp`.



Figura 2: Imagen de entrada `input.bmp` en formato BMP de 24 bits (BGR).

7.2. Salida en consola

Tras compilar en **Release** | **x64** y ejecutar el programa, la consola muestra los tres mensajes del flujo:

```
Consola de depuración de M6: x + -
Comprobando existencia de src/input.bmp...
Convirtiendo a escala de grises...
Exito: Se ha logrado la salida en src/output.bmp

C:\Users\ING_EJCA\Desktop\Proyecto01_Grayscale\proyecto\x64\Debug\Proyecto01_Grayscale.exe (proceso 9232) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->Cerrar la consola a
utomáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . . |
```

Figura 3: Mensajes en consola del programa: validación, conversión y aviso de éxito.

7.3. Imagen de salida en escala de grises

El archivo `src/output.bmp` contiene la misma imagen con los tres canales B, G y R sustituidos por el valor de gris calculado por `grayscale.asm`. Las cabeceras BMP/DIB y el padding de cada fila se conservan intactos, por lo que la imagen sigue siendo un BMP válido de 24 bits.



Figura 4: Imagen de salida `output.bmp` convertida a escala de grises usando los pesos ITU-R BT.601.

7.4. Inspección hexadecimal de las cabeceras

Para verificar que la estructura binaria del archivo es la esperada, se abre el BMP de entrada con un editor hexadecimal (HxD, GHex, etc.):

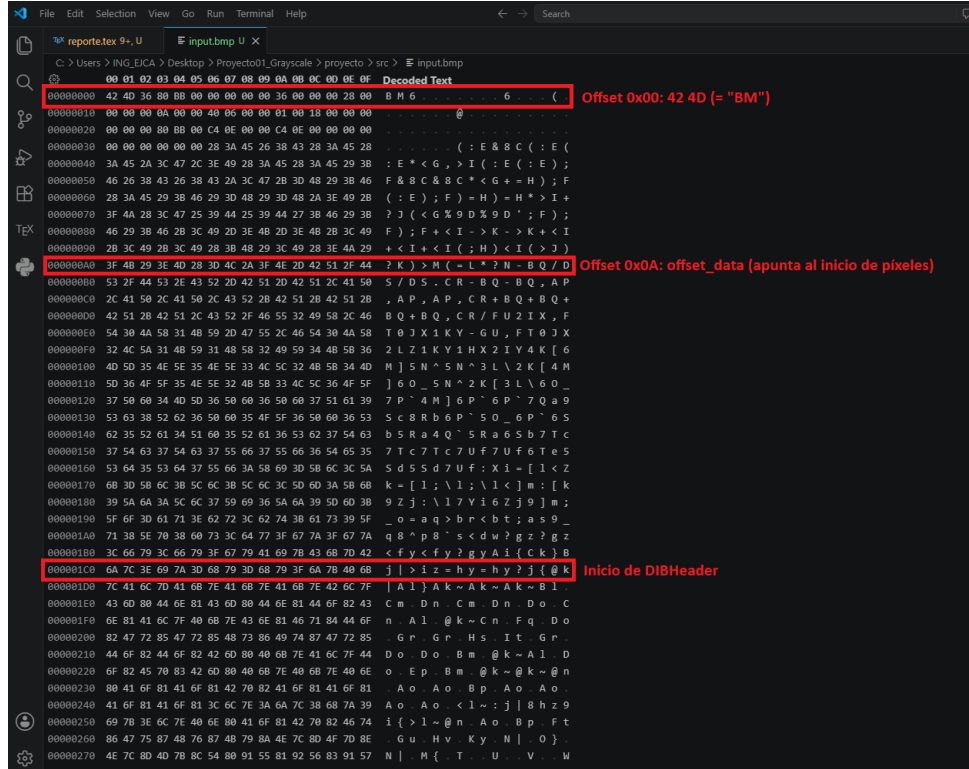


Figura 5: Vista hexadecimal de `input.bmp`. Los primeros bytes 42 4D corresponden a la firma ASCII "BM"; en el offset 0x1C aparece 18 00 (= 24 en little-endian), confirmando que es un BMP de 24 bits por píxel.

8. Repositorio GitHub

https://github.com/7mo-ArquitecturaComputadoras/Proyecto01_Grayscale

9. Conclusiones

- El formato **BMP de 24 bits** es un excelente vehículo didáctico porque expone explícitamente todos los detalles de bajo nivel que el procesador requiere: orden de bytes (BGR), alineación de filas a 4 bytes, almacenamiento bottom-up y la diferencia entre estructuras empaquetadas (`#pragma pack`) y alineadas naturalmente [1, 4].
- La fórmula de luminancia **ITU-R BT.601** $Y' = 0,299R + 0,587G + 0,114B$ no es arbitraria: refleja la sensibilidad espectral del ojo humano y produce escalas de grises perceptualmente correctas.

- Escalar los pesos por 2^8 permite implementar la fórmula con **aritmética entera**, sustituyendo la división por una instrucción **SHR** de un solo ciclo. La precisión perdida es despreciable porque el error de redondeo máximo por píxel es de ± 1 en una escala de 256 niveles.
- La elección de la convención **Microsoft x64** obliga a usar **RCX**, **RDX** y **R8** para los tres parámetros, lo que es notablemente más eficiente que el modelo **cdecl** de 32 bits porque evita lecturas/escrituras a la pila para cada llamada [6].
- Procesar la imagen **fila por fila** en C++ y delegar a ensamblador solo el bucle interno es un compromiso pragmático: el código en C++ maneja la complejidad del formato (cabeceras, padding, archivos) mientras que el ensamblador se concentra en el trabajo verdaderamente intensivo. Este patrón es común en bibliotecas de procesamiento de imágenes profesionales.

Referencias

- [1] Microsoft Corporation, “Bitmap Storage,” *Windows GDI Documentation*. <https://learn.microsoft.com/en-us/windows/win32/gdi/bitmap-storage>
- [2] Microsoft Corporation, “BITMAPINFOHEADER structure,” *Windows GDI Reference*. <https://learn.microsoft.com/en-us/windows/win32/api/wingdi/ns-wingdi-bitmapinfoheader>
- [3] Microsoft Corporation, “BITMAPINFO structure,” *Windows GDI Reference*. <https://learn.microsoft.com/en-us/windows/win32/api/wingdi/ns-wingdi-bitmapinfo>
- [4] “BMP File Format Specification,” *File Format Documentation*. <https://docs.fileformat.com/image/bmp/>
- [5] Intel Corporation, *Intel® 64 and IA-32 Architectures Software Developer’s Manual, Volume 2: Instruction Set Reference*. Santa Clara, CA, 2024. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- [6] Microsoft Corporation, “x64 calling convention,” *Microsoft C++ Documentation*. <https://learn.microsoft.com/en-us/cpp/build/x64-calling-convention>
- [7] “Data structure alignment,” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Data_structure_alignment